A 4x4 INT8 Systolic-Array Matrix Multiplier, from SystemVerilog to a Signed-Off Layout

Designed, verified, synthesized, placed, routed and checked in SkyWater 130 nm with open-source tools

Arjanveer Samra

What this project is, and why I built it

Matrix multiplication is most of the arithmetic in a neural network, and the hardware that does it in accelerators from Google's TPU to the tensor cores in a GPU is some version of a systolic array: a grid of small multiply-accumulate units that pass operands to their neighbours every clock, so each number fetched from memory is reused across a whole row or column. My RV32I project taught me how a design goes from Verilog to a layout. I wanted to do the same for the kind of block that AI hardware is actually built from, and to learn where its time, area and energy go.

So I designed a 4x4 array of INT8 multiply-accumulate units with 32-bit accumulators, verified it against an independent reference model, and took it through synthesis, placement, routing, extraction, three-corner timing closure, DRC and LVS in the SkyWater 130 nm open process. Because the RTL is parameterized I also implemented a 2x2 and an 8x8 version to see how the design scales.

Three things shape the rest of this report. A mutation test showed that a latency I had derived for the control logic was wrong by a factor of two, in the safe direction. The first layouts missed timing at the slow corner for a reason that had nothing to do with the clock period. And getting LVS to pass took five separate fixes, four of them in the checking flow rather than the design, each of which I had to prove was a real problem and not me weakening the check.

The short version

Result	Value	Source
Clock closed at all three PVT corners (4x4)	100 MHz (10.0 ns)	SIMULATED: OpenSTA, ss/tt/ff, OpenRCX SPEF
Same layout, typical corner only	159.1 MHz	CALCULATED from tt slack
Setup slack, slow corner	+0.0713 ns	SIMULATED (ss 100 C 1.60 V)
Hold slack, fast corner	+0.0952 ns	SIMULATED (ff -40 C 1.95 V)
Peak throughput	3.2 GOPS (16 MAC/cycle)	CALCULATED
Power, random INT8 workload	17.14 mW	SIMULATED: VCD activity from gate-level sim, tt
Energy per MAC	16.1 pJ	CALCULATED from simulated power
Die size	444.4 x 444.4 um	layout
Standard-cell area	92,470 um^2	layout (before fill)
Results checked against reference model	115,164 over 4,113 tiles, 0 mismatches	SIMULATED: cocotb + Icarus, RTL, N=2/4/8
Gate-level runs (synthesized + routed netlists)	pass, 0 mismatches	SIMULATED: Icarus, sky130 cell models
Design rule check	0 violations (63 found and fixed)	KLayout, SkyWater sky130A_mr deck
Layout versus schematic	Netlists match	KLayout, SkyWater LVS deck
Antenna nets still reported	1	OpenROAD check_antennas
8x8 variant	closes all corners at 11.0 ns	SIMULATED

Every number in this report is read from one results file (params.json) that the scripts write; nothing is typed by hand. CALCULATED means arithmetic on simulated values; SIMULATED names the tool and corner.

1. The tools, and getting them to run

The RV32I project ran OpenROAD as a Linux x86-64 Python wheel inside Docker. This Mac has no Docker, about 15 GB of free disk and an Apple-silicon CPU, and there is no macOS build of that wheel, so every tool here runs natively. Yosys, Icarus Verilog and Verilator came from Homebrew; cocotb runs in a Python 3.12 virtual environment; KLayout, netgen and the PDK were already built for my SAR ADC project. OpenROAD I built from source, which took five fixes:

- CUDD 3.0.0 is not packaged by Homebrew, so I built it from source.
- Boost 1.92's stacktrace header refuses to compile on macOS without BOOST_STACKTRACE_GNU_SOURCE_NOT_REQUIRED.
- OpenROAD's SWIG interface files use Python 2 C-API names that SWIG 4.5 no longer provides, so I built without the Python bindings and ported my RV32I flow script from the Python API to Tcl. Same steps, same platform scripts.
- The bundled SystemVerilog frontend includes fmt/core.h, but Homebrew's fmt 12 moved fmt::format to fmt/format.h; I patched the include in the build tree.
- The linker needed /opt/homebrew/lib for zstd.

The version I built is newer than the one RV32I used, and its timing engine (OpenSTA 3.1.0) replaced per-corner "-corner" report arguments with "-scenes". That broke my first flow run on the first report command, which is the right way for it to break.

Tool	Version
Yosys	0.69
Icarus Verilog	13.0
Verilator (lint)	5.052
cocotb	2.1.0
OpenROAD	git b931a33
OpenSTA	3.1.0
KLayout	0.30.12
PDK	sky130A open_pdks 0fe599b

Table 1. Tool versions.

2. Architecture

2.1 Output-stationary, because nothing large has to move

In an output-stationary array each processing element (PE) owns one result C[i][j] and keeps its 32-bit accumulator in place for the whole tile. Row i of A streams in from the left and column j of B from the top, one value per clock, and each PE passes what it received to its right and lower neighbour. The alternative, weight-stationary (the TPU's choice), preloads B into the PEs and moves 32-bit partial sums down the columns every cycle. For a small array fed by a simple stream, output-stationary moves only 8-bit operands between PEs, needs no preload phase, and lets the reduction length K be anything: the stream marks its last beat.

2.2 Interface

Both ports are valid/ready streams in the AXI-Stream style. One input beat carries column k of A and row k of B (two times four bytes) plus a last flag. The output port delivers the four rows of C, each as four 32-bit results, with a last flag on the final row. Back-pressure on either side simply stalls the stream.

2.3 The processing element

Each PE has two pipeline stages: the 8x8 signed product is registered, then added into the accumulator. A "first" flag travels with the operands and marks k = 0, so the accumulator is loaded instead of added to and no clear cycle is needed between tiles. Operands and the flags travel along rows; because row i is delayed by i cycles at the edge and column j by j cycles, A[i][k] and B[k][j] meet in PE(i,j) on the same clock, so the flags that ride with A are valid for B too. Datapath registers have no reset; only the control flags do.

2.4 Control, and a latency I got wrong

The controller has three states: LOAD accepts beats, DRAIN waits for the last products to reach every accumulator, and READ hands out the four rows. A new tile is accepted only after the last row has been read, so a result can never be overwritten before it leaves. My first version drained for 2N cycles, the time for the last operand to reach the far corner PE(N-1,N-1). Section 3.4 explains how a mutation test showed that was twice as long as it needs to be; the final design drains for N cycles. A tile of length K therefore takes K + 2N cycles with no stalls: K to load, N to drain, N to read out.

Parameter	Value
Array	4x4 (parameter N; also built as 2x2 and 8x8)
Operands / accumulators	8-bit signed / 32-bit
Dataflow	output-stationary
PE pipeline	2 stages
Input beat / output beat	64 bits / 128 bits
Cycles per tile, no stalls	K + N (drain) + N (readout), no stalls
Peak MACs per cycle	16

Table 2. Architecture as committed.

3. Verification

3.1 A reference model that knows nothing about timing

The golden model is plain integer arithmetic: C = A x B with INT8 operands and a two's-complement 32-bit wrap. It has no notion of skew, pipelines or handshakes, which is the point: it can only agree with the RTL if the RTL computes the right numbers.

3.2 The testbench

A cocotb testbench streams every tile through the real valid/ready ports and compares all N x N results of every tile with the model. It runs directed cases (identity, zeros, a single-beat tile, the longest tile of all -128 so the accumulator reaches its largest value, alternating extremes) and then hundreds of random tiles with K drawn from 1 to 256. Both streams are throttled at random, input bubbles and output back-pressure, so the control path is exercised and not just the arithmetic. During every input bubble the testbench drives random garbage on the operand buses and the last flag, because the final RTL lets operands through ungated (Section 5.3) and relies on the valid flag alone.

Array	Tiles	Results checked	Distinct K values	Mismatches
2x2	1,371	5,484	154	0
4x4	1,371	21,936	145	0
8x8	1,371	87,744	147	0
Total	4,113	115,164		0

Table 3. RTL regression, three seeds per array size. SIMULATED (Icarus Verilog 13.0 + cocotb 2.1.0, RTL).

3.3 Two bugs in the testbench, not the design

- The first run hung. The testbench sampled the handshake after the clock edge, by which time the array had accepted the beat and left LOAD, so the test never saw the transfer and kept resending it. A transfer happens on the edge when valid and ready are both high before it, and that is where the test now samples.
- A directed vector computed (-1) x (-128) = +128, which is not an INT8 value. The golden model's range assertion caught it before the RTL was ever blamed.

3.4 Mutation testing, and the latency that was too long

A passing testbench only means something if it would fail on a broken design, so I broke the design on purpose and checked that the tests noticed:

- Accumulator ignores the first flag (never reloads between tiles): every test fails.
- Drain one cycle shorter than my original 2N: **all tests still passed.**
- Drain one cycle shorter than N: 233 / 432 / 755 mismatching results at N = 2 / 4 / 8.

The second result looked like a hole in the testbench. It was not: my derivation was wrong. The first skew stage is combinational, so PE(r,c) registers its final accumulator r + c + 1 edges after the last beat is accepted, and readout presents row r at edge DRAIN + r. Row r is complete when c = N-1, at r + N, so DRAIN = N is enough, and N - 1 fails at every size, which the third mutant confirms. The original 2N waited for the whole wavefront to finish before reading row 0, when row 0 is done long before row N-1. Shortening it saves N cycles per tile.

3.5 Checking the netlists, not just the RTL

The same cocotb regression runs on the synthesized netlists of all three sizes and on the routed netlists of all three sizes, using SkyWater's functional cell models. The routed netlist includes everything the physical tools added: resizer buffers, the clock tree and antenna diodes. All pass with zero mismatches.

Netlist	Array	Tiles	Results checked	Mismatches
synth	2x2	207	828	0
synth	4x4	207	3,312	0
synth	8x8	207	13,248	0
routed	2x2	207	828	0
routed	4x4	207	3,312	0
routed	8x8	157	10,048	0

Table 4. Gate-level simulation. SIMULATED (Icarus 13.0 + cocotb 2.1.0, functional sky130_fd_sc_hd models, unit delay).

4. Synthesis

Yosys maps the design to the sky130_fd_sc_hd library with ABC. I map against the slow-corner library rather than the typical one, because the slow corner is where closure is decided; Section 5.3 explains how I learned that. The RTL is read once and the array size is set by a macro on a small fixed-size top level, because Yosys does not implement SystemVerilog's ".*" port shorthand: my first wrapper left every output undriven, and Yosys's own consistency check refused it.

Array	Cells	Flip-flops	Area (um^2)	MACs per cycle
2x2	2,084	253	19,032	4
4x4	8,458	1,079	75,755	16
8x8	33,365	4,457	303,452	64

Table 5. Synthesis results. SYNTHESIZED (Yosys 0.69 + ABC, mapped against ss_100C_1v60, -D 9000 ps).

5. Physical design

5.1 Settings

Placement and routing follow the RV32I flow stage for stage: floorplan, pin placement, tap cells and the platform power grid, global placement, electrical repair, detailed placement, clock-tree synthesis, setup and hold repair, global routing, repair again against routed parasitics, detailed routing, fill, then OpenRCX extraction and signoff timing. The difference is that this flow is multi-corner from the first command: the slow, typical and fast libraries are all loaded before anything is placed, so every repair step, hold repair in particular, sees all three corners at once. The RV32I project learned that the hard way, when a design repaired at one corner failed hold at another.

Inputs and outputs are each budgeted 20% of the period, as if the array sat between other on-chip logic; clock uncertainty is 0.25 ns, and the floorplan starts at 40 % utilization.

5.2 Finding the frequency

I implemented the 4x4 at six clock periods, each a complete run from floorplan to extracted timing, and looked at the worst setup slack at each corner. The slow corner crosses zero between 9.5 and 10 ns, so the frequency this layout can promise across process, voltage and temperature is 100 MHz. The same layout has +3.7134 ns of setup slack at the typical corner, which would allow 159.1 MHz at the typical corner. Above 10 ns the slow-corner curve flattens instead of rising, because the timing repair only works until it reaches its target margin; it does not chase slack nobody asked for.

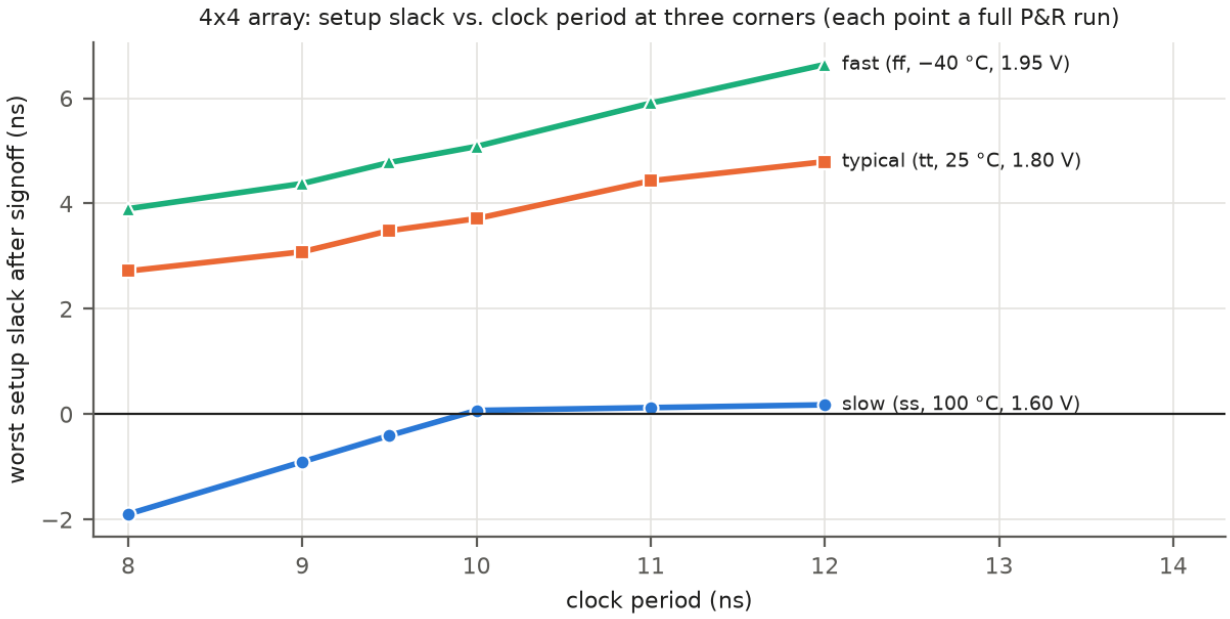


Figure 1. Worst setup slack after signoff at each corner versus clock period, 4x4 array.

5.3 A slow-corner miss that was not about the clock

My first 4x4 run missed the slow corner by -0.0371 ns at 10 ns. Lengthening the period to 10.5 ns still missed, by -0.0509 ns. A path that misses by about the same amount at two periods is not a period problem, so I looked at what happened between stages. At 10 ns the slow-corner slack after global routing was -1.7301 ns, the post-route repair pulled it to -0.0790 ns, and the real extraction then moved it by +41.9 ps; at 10.5 ns the same three numbers were -0.3972 ns, -0.0164 ns and -34.5 ps. The resizer stops when it reaches zero against its own parasitic estimate, so the final result is decided by the tens of picoseconds between that estimate and the extraction. Three changes fixed it:

- A 0.15 ns setup margin and 10 % slew and capacitance margins on every repair, so the repair aims past zero and the estimate-to-extraction gap no longer decides the result.
- Mapping against the slow-corner library in synthesis instead of the typical one.
- An RTL change. The input skew stage zeroed the operands whenever no beat was accepted, which put in_valid in front of a multiplexer as wide as both operand buses, and that was one of the failing paths. The zeroing did nothing useful, because a PE already ignores operands whose valid flag is low, so I removed it and made the testbench drive garbage during bubbles to prove the claim.

5.4 Three corners on the final design

Array	Period	Setup ss	Setup tt	Setup ff	Hold ss	Hold tt	Hold ff	All corners
2x2	10.0 ns	+0.0396 ns	+3.7793 ns	+5.2395 ns	+0.7820 ns	+0.2798 ns	+0.0955 ns	closed
4x4	10.0 ns	+0.0713 ns	+3.7134 ns	+5.0837 ns	+0.7481 ns	+0.2770 ns	+0.0952 ns	closed
8x8	11.0 ns	+0.0474 ns	+3.9112 ns	+5.4702 ns	+0.2542 ns	+0.2674 ns	+0.0920 ns	closed

Table 6. Signoff slack per corner after OpenRCX extraction. One typical RC extraction is used at all three library corners.

5.5 Where the energy goes

A vectorless power estimate assumes every input toggles 10 % of the time (the OpenSTA default), which says little about a datapath. Instead I ran the routed netlist in Icarus on a realistic workload, back-to-back tiles of uniformly random INT8 values with K = 16, dumped the switching activity, and annotated it onto the extracted design in OpenSTA. The 4x4 draws 17.14 mW at the typical corner with that activity, against 26.09 mW vectorless. The simulated window ran at 24.0 cycles per tile, exactly the K + 2N the architecture predicts, so 66.7% of the MAC slots did useful work, and the energy works out to 16.1 pJ per multiply-accumulate.

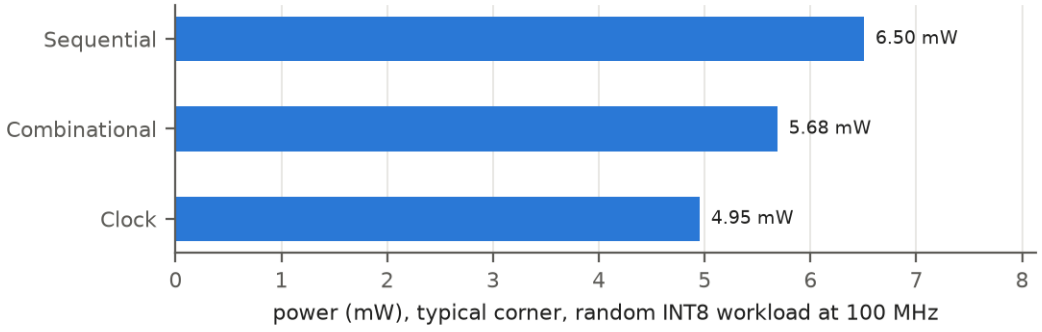


Figure 2. Power by group, 4x4, typical corner, VCD activity from the routed netlist.

Registers and the clock tree together use more than the arithmetic. Each PE holds 8-bit forwarding registers for A and B, a 16-bit product register and a 32-bit accumulator, and every one of them is clocked every cycle whether or not it changes. Clock gating the accumulators while a PE waits would be the first thing to try.

5.6 What the placer did with the array

I expected each PE to come out as a compact block. It did not. Colouring the 32 accumulator flip-flops of each PE in the routed placement shows each accumulator laid out as a strand, the carry chain stretched into a line, and the four PEs of a column running side by side in one region. The numbers agree: the flip-flops of one PE are spread over an RMS radius of 114 um, almost the same as a whole column of four PEs (118 um) and much tighter than a row (157 um). The reason is the readout: output lane j of the result port takes accumulator [r][j] from every row r, so the pins of lane j pull a column together, while the systolic neighbour wires are only 8 bits wide and lose the argument.

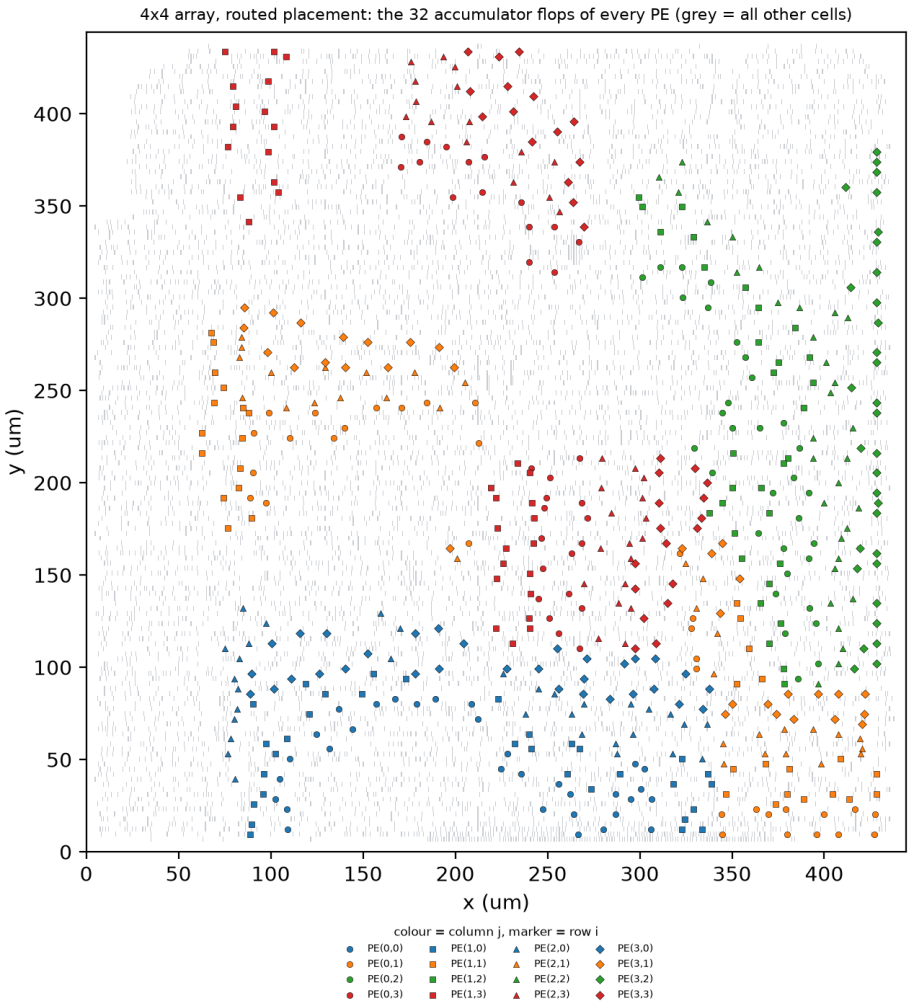


Figure 3. Routed placement of the 4x4: the accumulator flip-flops of each PE (colour = column, marker = row).

Quantity	2x2	4x4	8x8
Die (um x um)	227.3 x 227.3	444.4 x 444.4	880.2 x 880.2
Standard-cell area (um^2)	23,322	92,470	375,157
Placed instances incl. fill	7,380	29,764	117,184
Routed wire (um)	60,968	275,676	1,323,411
Router DRC	0	0	0
Max slew / max cap violations	0 / 0	0 / 0	143 / 7
Antenna nets reported	0	1	5

Table 7. Physical results of the signoff runs.

6. Defects found and fixed

Not all of these are design bugs. Some are in my testbench, some in my flow, some in tools. I list them all because each one would have produced a wrong or false result if I had not chased it.

#	What happened	Where	How it was caught	Fix
1	Test hung: handshake sampled after the edge	testbench	hang at 0 ns, isolated with a plain Verilog bench	sample before the edge
2	Directed vector produced +128	testbench	golden-model range assertion	use 127 / -128
3	Drain latency 2N instead of N	RTL (performance)	mutation test that should have failed did not	derive, set N, prove N-1 fails
4	Operand zeroing on the in_valid fanout	RTL (timing)	slow-corner path analysis	removed; garbage-bubble tests
5	SystemVerilog .* left outputs undriven	synthesis	Yosys check -assert	explicit ports
6	Repair stopped at ~0 ps, extraction worse	flow	same miss at two periods	setup/slew/cap margins
7	Slew violations and antenna nets left after routing	flow	signoff checks report	post-route repair_design, repair_antennas twice
8	Diodes inserted after the power grid had floating supply pins	flow	LVS	final global_connect
9	GDS export used LEF abstracts, no transistors	flow (KLayout 0.30 semantics)	LVS extracted nothing	macro_resolution_mode 0
10	Reference netlist could not match multi-finger cells	signoff reference	LVS per-cell report	PDK layout-extracted SPICE
11	Detailed router crash on one 8x8 run	OpenROAD	segfault in GuideProcessor	documented; reran
12	8x8 missed ss by tens of ps at three periods	flow	same signature as #6	larger setup margin for the 8x8

Table 8. Defects found during the project.

7. Physical verification

7.1 Design rule checking

I streamed the routed design to GDSII and checked it with SkyWater's full KLayout deck (front end, back end and off-grid). OpenROAD's router reported zero violations. The deck found 63 in the 4x4, all the same rule as in my RV32I project: m3.6, isolated met3 landing pads under the 0.24 um^2 minimum area. I reused the repair step from that project, in its final form with all three of its own bugs fixed (Section 7 of the RV32I report): each pad grows along its longer axis, on one side only, in whole 5 nm grid steps, and only if the grown shape keeps full spacing to every other met3 shape including the pads not yet patched. 63 of 63 were patched and none skipped; the deck then reports 0.

7.2 Layout versus schematic, and five reasons it failed

LVS passed only after five fixes. Every one changes how the comparison is set up, never what it is allowed to ignore, and the last step below is the proof:

- **No transistors in the GDS.** My export script set KLayout's macro_resolution_mode to 1 with a comment saying it takes cell geometry from the foundry GDS. In KLayout 0.30 mode 1 means always use the LEF abstract, so the layout had pins and labels but no diffusion or poly, and a single li.6 "violation" in a via cell was an artifact of that. Mode 0 uses the real cell layouts.
- **The deck could not run on macOS.** Its log formatter calls the Linux tool pmap. I keep a copy of the deck with that line changed; the report directory has the two-line diff.

- **The reference netlist did not describe the layout.** The cell CDL I started from gives multi-finger cells such as a211oi_4, a21oi_2 and o211ai_4 different internal nodes from their layouts, so they could never match. I switched to the PDK's SPICE netlists, which are extracted from the cell layouts, rewrote their device calls as transistor elements, mapped special_nfet_01v8 to the nfet_01v8 the deck extracts, and dropped the antenna-diode junction, which the deck does not extract. I had first excluded o211ai_4 from the design for this reason; it stays excluded, but the real cause was the reference.
- **Floating diodes.** Antenna diodes inserted after the power grid never had their supply pins connected in the database, so the netlist showed them floating while the layout tied them to the rails. A final global_connect fixed the database.
- **Substrate and fill.** The deck names the substrate sky130_gnd; the taps tie it to VSS, so I named it VSS. Its "purge" option deleted the device-less tap cells and with them that tie, so I added a switch that combines parallel devices on both sides and flattens the fill cells instead of purging anything.

The result is "Congratulations! Netlists match." at all three sizes. Because four of those five fixes were to the checker, I then broke things on purpose. Swapping the clock and data nets of one flip-flop in the reference makes LVS fail; deleting one inverter from the layout makes LVS fail. My first canary swapped the two inputs of a NAND gate and LVS passed it, correctly, because those inputs are interchangeable; a canary has to be asymmetric.

7.3 What this does and does not prove

The chain holds in one direction. The RTL matches an independent model on every result of 4,113 tiles at three array sizes; the synthesized and routed netlists reproduce it; the layout matches the routed netlist under LVS; the layout passes the foundry rule deck; and all three sizes close setup and hold at three corners.

What it does not prove is that this could be manufactured. Timing uses three library corners but one typical RC extraction. There is no metal fill or density check, no I/O ring or seal ring, and the block is a macro, not a full chip. Power comes from one synthetic workload. The project ends at a signed-off layout, and every result here comes from simulation, static timing or physical verification.

8. Scaling: 2x2, 4x4 and 8x8

Because the RTL is parameterized, building the other sizes cost almost nothing, so I did. Area per PE barely moves, which says the controller and the input skew registers are a small overhead even at 2x2. Energy per MAC rises slightly with size, as wires get longer and the clock tree grows. The typical-corner frequency of the layout holds at the 4x4 and drops at the 8x8, where the same accumulator carry path has to cross a die four times the area.

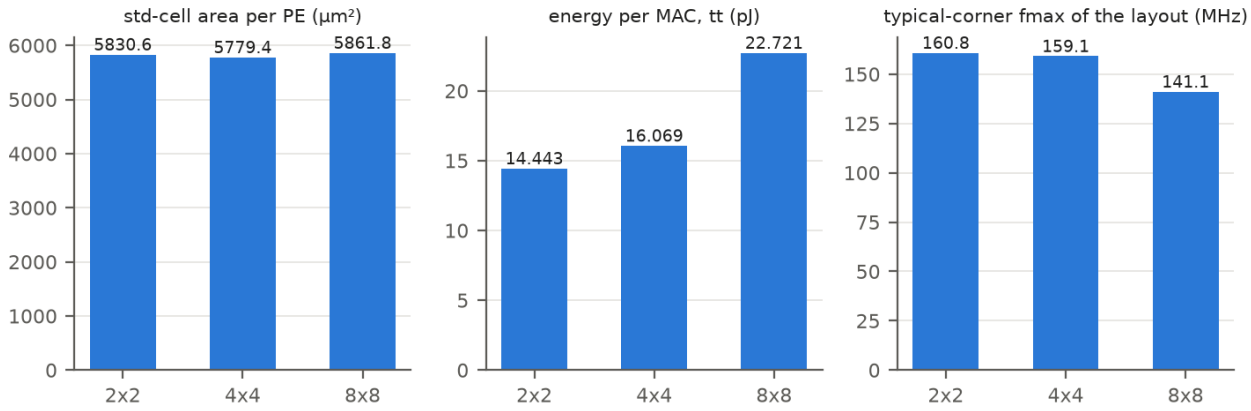


Figure 4. Scaling of area per PE, energy per MAC and typical-corner frequency of each layout.

The 8x8 is the one place where the three-corner discipline changes the answer. At 10 ns it closes at the typical and fast corners but misses the slow corner by -0.5870 ns. Lengthening the clock to 11, 11.5 and 12 ns with the same 0.15 ns repair margin still missed, by -0.0372 ns, -0.0911 ns and -0.0567 ns: the same signature as the first 4x4 runs, a repair that stops at its target and an extraction that lands a little worse, only larger because the 8x8 has four times the wire. With a 0.35 ns margin it closes all three corners at 11.0 ns, 90.9 MHz. The margin a flow needs grows with the design. (The first 12 ns attempt also crashed inside OpenROAD's detailed router; the identical rerun completed.)

Quantity	2x2	4x4	8x8
PEs	4	16	64
Signoff period (ns)	10.0	10.0	11.0
Closed all corners	yes	yes	yes
Peak GOPS at that period	0.80	3.20	11.64
Effective GOPS, K = 16	0.64	2.13	5.82
Power, tt, VCD (mW)	4.63	17.14	66.30
Energy per MAC, tt (pJ)	14.44	16.07	22.72
Std-cell area per PE (um^2)	5,831	5,779	5,862
Die area (mm^2)	0.0517	0.1975	0.7747

Table 9. Scaling summary (CALCULATED from SIMULATED values).

9. Discussion and limitations

9.1 Where the area and time go

The design is registers. Every PE carries forwarding, product and accumulator registers, and flip-flops are among the largest cells in the library, which is why registers and the clock tree lead the power breakdown. The critical path at every size is the accumulate: the accumulator register, about twenty levels of carry logic and the first-flag multiplexer, back into the accumulator. Splitting that add, for example carry-save accumulation with a final resolve during drain, is the change that would raise the frequency most.

9.2 Everything that is still wrong with it

- The array idles during drain and readout: at K = 16 the 4x4 does useful work in 66.7% of its MAC slots. Double-buffering the accumulators would let the next tile load while the last one drains.
- Timing uses one typical RC extraction at all three library corners; a real signoff extracts RC corners too.
- The 8x8 closes all corners only at 11.0 ns, and still reports 143 max-slew and 7 max-capacitance violations after repair.
- 1 antenna net(s) remain on the 4x4 after two rounds of diode insertion.
- No metal fill, density or seal ring; a macro, not a chip. Power is from one synthetic workload.
- Two local changes to the SkyWater LVS deck (macOS logging, a combine-without-purge switch), documented and checked with canaries, but a reviewer should know they exist.

9.3 If I kept going

Carry-save accumulators, double-buffered outputs, clock gating on idle accumulators, and a weight-stationary variant at the same size to compare the two dataflows on the same tools. Then a small RISC-V host on the same die, which would join this project to my RV32I one.

10. Reproducing it

Everything runs from the project folder after `source scripts/env.sh`:

```
make -C tb N=4 NTILES=400 SEED=1   (RTL regression)
LIB_FILE=$LIB_SS OUT_DIR=synth/out_n4 CLK_PERIOD_PS=9000 ARRAY_N=4 yosys -c synth/synth.tcl
python3 scripts/run_pnr.py 4 10 final_n4_10ns; python3 scripts/collect_pnr.py final_n4_10ns
python3 scripts/stage6_signoff.py final_n4_10ns --full; python3 scripts/stage6_canary.py final_n4_10ns
python3 scripts/stage5_power.py final_n4_10ns; python3 scripts/stage7_summary.py
python3 scripts/stage8_figures.py; python3 scripts/stage8_report.py; python3 scripts/stage10_selfcheck.py
```

HANDOFF.md in the same folder records every decision above with its reasoning.